

Dynamic Consistency Boundaries vs. Multi-Aggregate-Append Transactions

Prepared: June 11, 2026

Summary

One of the more interesting architectural tensions in event-sourced systems is between Multi-Aggregate-Append transactions and Dynamic Consistency Boundaries (DCB). They look similar on the surface because both deal with consistency across multiple domain concepts, but they differ in where they make the system pay for that consistency.

Multi-Aggregate-Append keeps streams or aggregates as the primary structural units, but lets a command write to several of them atomically. DCB shifts the consistency boundary away from fixed streams and toward the actual facts a command relied on: event types, tags, and an append condition over a relevant event query.

The strongest conclusion is not that one model replaces the other. It is that they optimize different failure modes.

The Basic Tension

Multi-Aggregate-Append transactions are attractive because they preserve a classic event-sourced aggregate model. A command can coordinate known streams, use optimistic concurrency on those streams, and commit all resulting events in one transaction.

This works well when the command naturally knows the streams it must coordinate: for example, transferring money between two accounts, updating an order while reserving inventory, or appending multiple correlated facts in one service boundary.

The concern is structural debt. Some business operations require facts from multiple concepts, but not necessarily because the domain wants those concepts to be separate transactional write targets. If the storage model forces the system to write to multiple aggregate streams merely to maintain consistency, streams can gradually stop feeling like clean autonomous units. Refactoring can also become harder, because moving events between streams requires migration logic and coordination.

DCB approaches the problem differently. Instead of asking which streams must be written, it asks which facts must not have changed since the command made its decision. Events can carry tags for several domain concepts, and a command can append with a condition over the relevant tagged events.

In that model, a single event such as `StudentSubscribedToCourse` can participate in both a student boundary and a course boundary without being duplicated into

two separate aggregate streams.

Why DCB Is Appealing

DCB is strongest when the aggregate boundary itself is the source of pain.

It lets the consistency boundary be defined by the command's decision criteria: all events of certain types tagged with a course, a student, an instructor, a room, a policy, or another domain concept. This can make consistency more semantic and less tied to whichever stream layout was chosen earlier in the project.

That has several advantages:

- One business fact can belong to multiple concepts through tags.
- Command consistency can evolve without physically moving events between streams.
- Cross-concept invariants can be expressed directly as event criteria.
- The write model can avoid artificial event duplication that exists only to satisfy aggregate-stream storage.

This is why DCB is sometimes described as moving from static consistency boundaries to dynamic ones. The boundary is not declared once as "this aggregate stream." It is computed from the facts relevant to the current decision.

The Critter Stack Objection

The Critter Stack perspective complicates the story. In Marten and Wolverine, the comparison is not simply "DCB flexibility versus rigid aggregates." Marten already gives developers mature projection and transaction options:

- Live projections are rebuilt from event data on demand.
- Inline projections are updated transactionally as events are appended.
- Async projections are built in the background and can serve scalable read or write models when eventual consistency is acceptable.
- Cross-stream or multi-stream command workflows can be optimized by the store.

From this angle, DCB can impose a cost. If a command handler must always search through raw events and project a decision model in memory, it may be doing a live projection for every decision. That can be fine for a handful of events, but less compelling when the system could fetch a prebuilt write model projection instead.

There is also code cost. DCB can encourage command-specific decision models and append conditions. That may align with a vertical slice architecture, but it can still mean more code, more projection definitions, more tag discipline, and more indexing concerns.

The critique is especially strong in Marten because the store already supports efficient write-model projections and strong consistency patterns across

streams. If a system can fetch an inline or optimized async projection and then atomically append to the relevant streams, Multi-Aggregate-Append may be more efficient and less complex than DCB for that use case.

Reframed Comparison

The useful comparison is not:

```
DCB flexible, Multi-Aggregate-Append rigid.
```

It is:

DCB:

- flexible semantic boundary
- boundary reconstructed from event criteria, tags, and append conditions

Multi-Aggregate-Append plus write projections:

- known structural coordination
- decision can use optimized prebuilt state

DCB pays with query design, tag discipline, indexing, and sometimes live projection work. Multi-Aggregate-Append pays with stream coupling, possible event duplication, and harder structural refactoring when the chosen stream layout no longer matches the domain's consistency needs.

Example: Student Course Subscription

Consider a command:

```
SubscribeStudentToCourse(studentId, courseId)
```

The command may need two invariants:

```
Course cannot exceed capacity.  
Student cannot subscribe to more than 10 courses.
```

With Multi-Aggregate-Append, the handler might read a student stream and a course stream, then append one event to each:

```
Student stream: StudentSubscribedToCourse  
Course stream: CourseSeatReserved
```

The transaction commits both or neither. This is direct and can be efficient, especially if prebuilt write projections already contain the current student and course state.

With DCB, the handler might read relevant events tagged with the student and the course, then append one event:

```
Event: StudentSubscribedToCourse  
Tags: student:s1, course:c1  
Append condition: no matching student/course subscription facts changed since  
the command's read position.
```

The DCB version avoids duplicating the same business fact into two streams and lets the event participate in multiple consistency boundaries. But it depends on efficient tag lookup and correct criteria. If those queries are expensive, or if each command needs a specialized decision model, the practical cost may outweigh the modeling benefit.

When Multi-Aggregate-Append Is the Better Fit

Multi-Aggregate-Append is usually a good fit when:

- The command naturally knows the streams it must coordinate.
- The aggregate model is mostly correct and only occasionally crosses streams.
- The store supports efficient multi-stream append and optimistic concurrency.
- Write-model projections can be fetched cheaply through inline or persisted lifecycle support.
- The team values familiar aggregate modeling and predictable operational behavior.

In Marten/Critter Stack systems, this option is often stronger than it appears in abstract DCB discussions because prebuilt write models can reduce or remove the need to scan and project raw events during command handling.

When DCB Is the Better Fit

DCB is usually a good fit when:

- The aggregate stream boundary is unstable or repeatedly wrong.
- One business event naturally belongs to multiple domain concepts.
- The system has many cross-concept invariants that do not map cleanly to fixed streams.
- Refactoring stream ownership would create migration burden.
- The store has strong support for indexed tags and conditional appends over event criteria.

DCB becomes especially valuable when the question is not "which streams do I write?" but "which facts must not have changed since this decision was made?"

Practical Takeaway

Multi-Aggregate-Append is not merely a halfway step toward DCB. DCB is not automatically the destination. They solve overlapping consistency problems at different layers.

Multi-Aggregate-Append optimizes known structural coordination. DCB optimizes evolving semantic coordination.

The right choice depends heavily on the event store, projection lifecycle, query and index support, command shape, and whether the system's real pain is runtime performance or model rigidity.

In systems with limited projection support, DCB can be liberating. In systems like Marten, where inline, async, live, and cross-stream workflows are already well supported, the value of DCB is more empirical and more situational.

Balanced Position

A fair final position is:

Multi-Aggregate-Append and Dynamic Consistency Boundaries solve overlapping consistency problems at different layers. Multi-Aggregate-Append optimizes known structural coordination; DCB optimizes evolving semantic coordination. The right choice depends on event-store capabilities, projection lifecycle, query and index support, and whether the dominant pain is performance or model rigidity.

Sources

- DCB specification and overview: <https://dcb.events/>
- Marten DCB concepts: <https://github.com/JasperFx/marten/blob/master/dcb-concepts.md>
- Jeremy Miller, "Higher Performance Dynamic Consistency Boundary Development

with Marten 9.0": <https://jeremydmiller.com/2026/05/25/higher-performance-dynamic-consistency-boundary-development-with-marten-9-0/>

- Jeremy Miller, Critter Stack projection lifecycle discussion:

<https://jeremydmiller.com/2023/12/04/building-a-critter-stack-application-web-service-query-endpoints-with-marten/>

- Marten aggregate projection documentation:

<https://martendb.io/events/projections/aggregate-projections.html>

- EventSourcingDB DCB best practices:

<https://docs.event sourcingdb.io/best-practices/dynamic-consistency-boundaries/>